

COMPTE RENDU

Programmation Système - TP6 - Threads et Synchronisation
3e année Cybersécurité - École Supérieure d'Informatique et du
Numérique (ESIN)
Collège d'Ingénierie & d'Architecture (CIA)

Étudiant : HATHOUTI Mohammed Taha

Filière : Cybersécurité

Année : 2025/2026

Enseignant : M. Noufel & M. ElAmrani

Date : 20 mars 2026

Table des matières

1	Exercice 1 – Recherche du maximum dans un tableau	2
1.1	Recherche séquentielle du maximum	2
1.2	Création et terminaison des NT threads	2
1.3	Recherche du maximum local par chaque thread	3
1.4	Protection du maximum global avec un mutex	3
2	Exercice 2 – Multiplication de matrices	4
2.1	Structure et découpage du travail	4
2.2	Mesure du temps d'exécution	5
2.3	Comparaison des performances	6
3	Exercice 3 – Gestion de stock (producteur/consommateur)	6
3.1	Version sans synchronisation – observation des race conditions	6
3.2	Version synchronisée – mutex et variables de condition	7
4	Conclusion	10

1 Exercice 1 – Recherche du maximum dans un tableau

L'objectif de cet exercice est d'implémenter une recherche parallèle du maximum dans un tableau de NE entiers en utilisant NT threads POSIX. Le programme est conçu de manière modulaire avec les fichiers `main.c`, `max.c/max.h` et `threads.c/threads.h`.

1.1 Recherche séquentielle du maximum

La première étape consiste à écrire un programme C qui recherche le maximum dans un tableau de NE entiers dont les valeurs varient de 0 à NE*100. Le tableau est rempli aléatoirement avec `rand()`, et la recherche est effectuée par une simple boucle itérative dans la fonction `chercher_max_sequentiel()` du fichier `max.c`.

Listing 1 – max.c – Recherche séquentielle

```
1 #include "max.h"
2
3 int chercher_max_sequentiel(int *tab, int taille) {
4     int max = tab[0];
5     for (int i = 1; i < taille; i++) {
6         if (tab[i] > max) {
7             max = tab[i];
8         }
9     }
10    return max;
11 }
```

Le thread principal initialise le tableau, appelle cette fonction et affiche le résultat. Une vérification des arguments est effectuée en entrée : NE et NT doivent être strictement positifs, et NT ne peut pas dépasser NE.

1.2 Création et terminaison des NT threads

La deuxième étape consiste à créer NT threads fils via `pthread_create()` et à attendre leur terminaison avec `pthread_join()`. Chaque thread reçoit en argument une structure `ThreadArgs` contenant un pointeur vers le tableau, les indices de début et de fin de sa sous-partie, ainsi que son identifiant.

Listing 2 – threads.h – Structure ThreadArgs

```
1 #ifndef THREADS_H
2 #define THREADS_H
3
4 #include <pthread.h>
5
6 typedef struct {
7     int *tableau;
8     int start;
9     int end;
10    int id;
11 } ThreadArgs;
```

```

12
13 void creer_threads(pthread_t *tids, ThreadArgs *args, int NT,
14                   int NE, int *tab);
15 void attendre_threads(pthread_t *tids, int NT);
16
17 #endif

```

Le découpage du tableau en NT tranches est effectué dans `creer_threads()` : chaque thread prend une tranche de taille `NE/NT`, le dernier thread absorbant les éléments restants en cas de division non entière.

1.3 Recherche du maximum local par chaque thread

Dans cette étape, chaque thread effectue la recherche du maximum dans sa sous-partie du tableau et affiche la valeur trouvée.

Listing 3 – `threads.c` – Fonction thread avec max local

```

1 static void *thread_fn(void *arg) {
2     ThreadArgs *a = (ThreadArgs *)arg;
3     int max_local = a->tableau[a->start];
4
5     for (int i = a->start + 1; i < a->end; i++) {
6         if (a->tableau[i] > max_local) {
7             max_local = a->tableau[i];
8         }
9     }
10
11     printf("[Thread %d] indices %d a %d --> max local = %d\n",
12           a->id, a->start, a->end - 1, max_local);
13
14     return NULL;
15 }

```

On remarque sur la capture ci-dessous que l'ordre d'affichage des threads n'est pas déterministe (Thread 1 peut s'afficher avant Thread 0), ce qui illustre bien la nature concurrente de l'exécution.

```

hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP6/ex1$ ./ex1 20 4
Tableau : 778 1198 841 951 656 957 1074 1910 326 640 1963 923 1616 1736 1601 1645 1009 1657 826 507
[Séquentiel] Max = 1963
[Thread 1] indices 5 à 9 --> max local = 1910
[Thread 0] indices 0 à 4 --> max local = 1198
[Thread 2] indices 10 à 14 --> max local = 1963
[Thread 3] indices 15 à 19 --> max local = 1657

```

FIGURE 1 – Exécution avec 4 threads – affichage des maxima locaux

1.4 Protection du maximum global avec un mutex

La dernière étape introduit une variable globale `global_max` partagée par tous les threads. Sans protection, deux threads pourraient lire et écrire simultanément cette variable, causant une *race condition*. Un mutex (`pthread_mutex_t`) est utilisé pour protéger la section critique de mise à jour.

Listing 4 – max.c – Déclaration du mutex et du maximum global

```

1 #include <pthread.h>
2 #include "max.h"
3
4 int global_max = 0;
5 pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

```

Listing 5 – threads.c – Section critique avec mutex

```

1 /* section critique : mise a jour du maximum global */
2 pthread_mutex_lock(&mutex);
3 if (max_local > global_max) {
4     global_max = max_local;
5 }
6 pthread_mutex_unlock(&mutex);

```

Le mutex garantit qu'un seul thread à la fois peut comparer et modifier `global_max`. Le thread principal attend la fin de tous les threads avec `pthread_join()` avant d'afficher le résultat global, puis libère le mutex avec `pthread_mutex_destroy()`.

```

hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP6/ex1$ ./ex1 20 4
Tableau : 771 887 1612 1946 937 1942 1210 764 298 1658 352 1986 1807 1407 803 1747 17 13 1619 860
[Séquentiel] Max = 1986
[Thread 0] indices 0 à 4 --> max local = 1946
[Thread 1] indices 5 à 9 --> max local = 1942
[Thread 3] indices 15 à 19 --> max local = 1747
[Thread 2] indices 10 à 14 --> max local = 1986
[Threads]   Max global = 1986

```

FIGURE 2 – Exécution avec 4 threads – maximum global = maximum séquentiel

On vérifie que le maximum global affiché par les threads correspond bien au maximum obtenu par la recherche séquentielle, ce qui valide la correction du programme.

2 Exercice 2 – Multiplication de matrices

L'objectif de cet exercice est d'implémenter la multiplication de deux matrices carrées **A** et **B** de taille $N \times N$ pour obtenir $C = A \times B$, en parallélisant le calcul avec NT threads POSIX, puis de comparer le temps d'exécution entre la version séquentielle et la version parallèle. Le programme est structuré de manière modulaire avec les fichiers `main.c`, `matrice.c/matrice.h` et `threads.c/threads.h`.

2.1 Structure et découpage du travail

Chaque élément `C[i][j]` de la matrice résultat est indépendant des autres, ce qui rend la parallélisation naturelle et sans risque de *race condition* sur la matrice résultat. Le travail est distribué par lignes : le thread `k` calcule les lignes `[start, end[` de **C**.

Listing 6 – threads.h – Structure ThreadArgs pour la multiplication

```

1 typedef struct {
2     double **A;
3     double **B;

```

```

4     double **C;
5     int N;
6     int start;
7     int end;
8     int id;
9 } ThreadArgs;

```

Listing 7 – threads.c – Fonction thread pour la multiplication

```

1 static void *thread_fn(void *arg) {
2     ThreadArgs *a = (ThreadArgs *)arg;
3
4     for (int i = a->start; i < a->end; i++) {
5         for (int j = 0; j < a->N; j++) {
6             double sum = 0.0;
7             for (int k = 0; k < a->N; k++) {
8                 sum += a->A[i][k] * a->B[k][j];
9             }
10            a->C[i][j] = sum;
11        }
12    }
13
14    printf("[Thread %d] lignes %d a %d terminees\n",
15           a->id, a->start, a->end - 1);
16    return NULL;
17 }

```

2.2 Mesure du temps d'exécution

Le temps d'exécution est mesuré en nanosecondes via `clock_gettime(CLOCK_MONOTONIC)` pour les deux versions séquentielle et parallèle. Deux matrices résultat distinctes `C_seq` et `C_par` sont utilisées afin de vérifier visuellement la cohérence des résultats pour les petites valeurs de `N`.

La capture ci-dessous montre la vérification manuelle avec `N=3` et `NT=2` : les matrices `A` et `B` sont affichées, et le résultat séquentiel est visible.

```

hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP6/ex2$ ./ex2 3 2
Matrice A :
  20.00    24.00    38.00
  22.00    24.00    94.00
  14.00    26.00    16.00
Matrice B :
  68.00    15.00    55.00
   5.00    52.00    13.00
  15.00    13.00    38.00
[Séquentiel] Temps = 227 ns
Matrice C (résultat) :
 2050.00  2042.00  2856.00
 3026.00  2800.00  5094.00
 1322.00  1770.00  1716.00

```

FIGURE 3 – Vérification manuelle avec `N=3` – matrices `A`, `B` et `C` séquentielle

2.3 Comparaison des performances

Pour des petites matrices ($N=3$), le temps parallèle dépasse largement le temps séquentiel en raison du coût de création des threads (`pthread_create()` implique des appels système et une allocation de pile). Le parallélisme devient avantageux uniquement lorsque le volume de calcul est suffisant pour amortir cet overhead.

Les captures suivantes illustrent ce comportement sur des matrices de grande taille.

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP6/ex2$ ./ex2 500 4
[Séquentiel] Temps = 285737591 ns
[Thread 2] lignes 250 à 374 terminées
[Thread 1] lignes 125 à 249 terminées
[Thread 3] lignes 375 à 499 terminées
[Thread 0] lignes 0 à 124 terminées
[Parallèle] Temps = 91808935 ns (4 threads)
```

FIGURE 4 – $N=500$, $NT=4$ – séquentiel : 285 737 591 ns, parallèle : 91 808 935 ns

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP6/ex2$ ./ex2 1000 4
[Séquentiel] Temps = 3757880934 ns
[Thread 1] lignes 250 à 499 terminées
[Thread 0] lignes 0 à 249 terminées
[Thread 2] lignes 500 à 749 terminées
[Thread 3] lignes 750 à 999 terminées
[Parallèle] Temps = 1180592583 ns (4 threads)
```

FIGURE 5 – $N=1000$, $NT=4$ – séquentiel : 3 757 880 934 ns, parallèle : 1 180 592 583 ns

On observe qu’avec $N=500$ et $NT=4$, le parallèle est environ **3 fois plus rapide** que le séquentiel. Avec $N=1000$, le gain est similaire (3,2x), ce qui confirme que le parallélisme est efficace pour des matrices de grande taille. On remarque également que l’ordre de terminaison des threads n’est pas déterministe, confirmant leur exécution réellement concurrente.

3 Exercice 3 – Gestion de stock (producteur/consommateur)

L’objectif de cet exercice est de simuler la gestion du stock d’un magasin avec un modèle producteur/consommateur : un thread magasin réapprovisionne le stock lorsqu’il devient trop bas, et `N_CLIENTS` threads clients consomment chacun un nombre aléatoire d’articles. La variable partagée critique est `store.valstock`. Le programme est structuré avec les fichiers `main.c`, `stock.c/stock.h` et `clients.c/clients.h`.

3.1 Version sans synchronisation – observation des race conditions

Dans cette première version, aucun mécanisme de synchronisation n’est utilisé. Plusieurs threads accèdent simultanément à `store.valstock` sans protection, ce qui provoque

des *race conditions* : un thread peut lire une valeur obsolète avant qu'un autre thread ait terminé sa mise à jour.

Listing 8 – clients.c – Accès non protégé à valstock

```
1 void *thread_client(void *arg) {
2     int id = *(int *)arg;
3     int prise = (rand() % 10) + 1;
4
5     printf("[Client %d] veut prendre %d articles (stock actuel :
6         %d)\n",
7         id, prise, store.valstock);
8
9     store.valstock -= prise;
10
11    printf("[Client %d] a pris %d articles (stock restant : %d)\n
12        ",
13        id, prise, store.valstock);
14
15    return NULL;
16 }
```

La capture ci-dessous illustre la race condition : Client 2 lit `valstock = 43` alors que Client 3 a déjà commencé à modifier cette valeur, produisant des affichages incohérents.

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP6/ex3$ ./ex3
[Main] Stock initial : 50
[Client 0] veut prendre 5 articles (stock actuel : 50)
[Client 0] a pris 5 articles (stock restant : 45)
[Client 1] veut prendre 2 articles (stock actuel : 45)
[Client 1] a pris 2 articles (stock restant : 43)
[Client 3] veut prendre 5 articles (stock actuel : 43)
[Client 3] a pris 5 articles (stock restant : 38)
[Client 2] veut prendre 2 articles (stock actuel : 43)
[Client 2] a pris 2 articles (stock restant : 36)
[Client 4] veut prendre 4 articles (stock actuel : 36)
[Client 4] a pris 4 articles (stock restant : 32)
[Main] Tous les clients ont terminé. Stock final : 32
```

FIGURE 6 – Version sans synchronisation – race condition visible entre Client 2 et Client 3

3.2 Version synchronisée – mutex et variables de condition

La version synchronisée utilise un **mutex** pour protéger tout accès à `store.valstock`, et deux **variables de condition** pour la coordination entre threads :

- `cond_stock_bas` : signale au thread magasin que le stock est passé sous le seuil `SEUIL_BAS`;
- `cond_stock_dispo` : signale aux clients en attente que le stock a été réapprovisionné.

Listing 9 – stock.h – Structure Store avec mutex et variables de condition

```

1 typedef struct {
2     int valstock;
3     pthread_mutex_t mutex;
4     pthread_cond_t cond_stock_bas;
5     pthread_cond_t cond_stock_dispo;
6 } Store;

```

Listing 10 – stock.c – Thread magasin avec attente sur variable de condition

```

1 void *thread_magasin(void *arg) {
2     (void) arg;
3     while (1) {
4         pthread_mutex_lock(&store.mutex);
5         while (store.valstock >= SEUIL_BAS) {
6             pthread_cond_wait(&store.cond_stock_bas, &store.mutex);
7         }
8         store.valstock += REAPPRO;
9         pthread_cond_broadcast(&store.cond_stock_dispo);
10        pthread_mutex_unlock(&store.mutex);
11    }
12    return NULL;
13 }

```

Listing 11 – clients.c – Client avec attente si stock insuffisant

```

1 void *thread_client(void *arg) {
2     int id = *(int *) arg;
3     int prise = (rand() % 10) + 1;
4
5     pthread_mutex_lock(&store.mutex);
6     while (store.valstock < prise) {
7         pthread_cond_wait(&store.cond_stock_dispo, &store.mutex);
8     }
9     store.valstock -= prise;
10    if (store.valstock < SEUIL_BAS) {
11        pthread_cond_signal(&store.cond_stock_bas);
12    }
13    pthread_mutex_unlock(&store.mutex);
14    return NULL;
15 }

```

La capture suivante montre la version synchronisée avec `N_CLIENTS = 5` : plus aucun chevauchement, chaque client lit et écrit des valeurs strictement cohérentes.

```

hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP6/ex3$ ./ex3
[Main] Stock initial : 50
[Client 0] prend 7 articles (stock actuel : 50)
[Client 0] a pris 7 articles (stock restant : 43)
[Client 2] prend 4 articles (stock actuel : 43)
[Client 2] a pris 4 articles (stock restant : 39)
[Client 1] prend 2 articles (stock actuel : 39)
[Client 1] a pris 2 articles (stock restant : 37)
[Client 3] prend 5 articles (stock actuel : 37)
[Client 3] a pris 5 articles (stock restant : 32)
[Client 4] prend 2 articles (stock actuel : 32)
[Client 4] a pris 2 articles (stock restant : 30)
[Main] Tous les clients ont terminé. Stock final : 30

```

FIGURE 7 – Version synchronisée avec 5 clients – cohérence totale des accès

Avec `N_CLIENTS = 15`, le stock descend sous `SEUIL_BAS = 10` à deux reprises, déclenchant deux réapprovisionnements de 30 articles par le thread magasin. Les clients en attente sont réveillés par `pthread_cond_broadcast()` après chaque réappro.

```

hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP6/ex3$ ./ex3
[Main] Stock initial : 50
[Client 0] prend 8 articles (stock actuel : 50)
[Client 0] a pris 8 articles (stock restant : 42)
[Client 1] prend 2 articles (stock actuel : 42)
[Client 1] a pris 2 articles (stock restant : 40)
[Client 3] prend 4 articles (stock actuel : 40)
[Client 3] a pris 4 articles (stock restant : 36)
[Client 2] prend 2 articles (stock actuel : 36)
[Client 2] a pris 2 articles (stock restant : 34)
[Client 4] prend 8 articles (stock actuel : 34)
[Client 4] a pris 8 articles (stock restant : 26)
[Client 5] prend 10 articles (stock actuel : 26)
[Client 5] a pris 10 articles (stock restant : 16)
[Client 6] prend 7 articles (stock actuel : 16)
[Client 6] a pris 7 articles (stock restant : 9)
[Client 7] prend 2 articles (stock actuel : 9)
[Client 7] a pris 2 articles (stock restant : 7)
[Magasin] Stock bas (7), réapprovisionnement de 30 articles
[Magasin] Stock après réappro : 37
[Client 9] prend 7 articles (stock actuel : 37)
[Client 9] a pris 7 articles (stock restant : 30)
[Client 8] prend 3 articles (stock actuel : 30)
[Client 8] a pris 3 articles (stock restant : 27)
[Client 10] prend 10 articles (stock actuel : 27)
[Client 10] a pris 10 articles (stock restant : 17)
[Client 11] prend 2 articles (stock actuel : 17)
[Client 11] a pris 2 articles (stock restant : 15)
[Client 12] prend 1 articles (stock actuel : 15)
[Client 12] a pris 1 articles (stock restant : 14)
[Client 13] prend 5 articles (stock actuel : 14)
[Client 13] a pris 5 articles (stock restant : 9)
[Magasin] Stock bas (9), réapprovisionnement de 30 articles
[Magasin] Stock après réappro : 39
[Client 14] prend 6 articles (stock actuel : 39)
[Client 14] a pris 6 articles (stock restant : 33)
[Main] Tous les clients ont terminé. Stock final : 33

```

FIGURE 8 – Version synchronisée avec 15 clients – deux réapprovisionnements déclenchés

4 Conclusion

Ce TP6 a permis de mettre en pratique les concepts fondamentaux de la programmation multithread sous Linux à travers trois exercices complémentaires.

Les points abordés sont les suivants :

- **Création et gestion de threads** : Utilisation de `pthread_create()` et `pthread_join()` pour lancer et synchroniser les threads fils depuis le thread principal ;
- **Exclusion mutuelle** : Protection des variables partagées via un mutex `pthread_mutex_t` pour éviter les conditions de concurrence ;
- **Variables de condition** : Coordination entre threads producteur et consommateurs via `pthread_cond_wait()`, `pthread_cond_signal()` et `pthread_cond_broadcast()` pour une synchronisation de type moniteur ;
- **Parallélisme sur les matrices** : Distribution des lignes de la matrice résultat entre les threads, sans nécessité de synchronisation puisque chaque thread écrit dans des cases distinctes ;
- **Analyse des performances** : Mise en évidence du seuil à partir duquel le parallélisme devient rentable — le coût de création des threads domine pour de petites entrées, mais le gain devient significatif (3x) pour des matrices de grande taille (N=500, N=1000) ;

L'ensemble des exercices illustre que la programmation multithread exige une gestion rigoureuse des accès concurrents : sans synchronisation, les résultats sont imprévisibles et incohérents, tandis qu'un usage correct des mutex et variables de condition garantit la correction des programmes.